COGS 181 Winter 2026 - Assignment 1

Instructor: Zhuowen Tu

- Assignment published on: **Tuesday, January 13, 2025**.
- Due on: **11:59pm, Tuesday, January 13 2025 (Pacific Time)**.

Getting Started with NumPy

`NumPy` is the fundamental package for scientific computing with Python. It provides a powerful N-dimensional array object and functions for working with these arrays. Some basic use of this packages is shown below. This is **NOT** a problem, but you are highly recommended to run the following code with some of the input changed in order to understand the meaning of the operations.

Arrays

In [33]:
```
import numpy as np             # Import the NumPy package

v = np.array([1, 2, 3])        # A 1D array
print(v)
print(v.shape)                 # Print the size / shape of v
print("1D array:", v, "Shape:", v.shape)

v = np.array([[1], [2], [3]])  # A 2D array
print("2D array:", v, "Shape:", v.shape) # Print the size of v and check the differ

# You can also attempt to compute and print the following values and their size.

v = v.T                        # Transpose of a 2D array
m = np.zeros([3, 4])           # A 3x4 array (i.e. matrix) of zeros
v = np.ones([1, 3])            # A 1x3 array (i.e. a row vector) of ones
v = np.ones([3, 1])            # A 3x1 array (i.e. a column vector) of ones
m = np.eye(4)                  # Identity matrix
m = np.random.rand(2, 3)       # A 2x3 random matrix with values in [0, 1] (sampled
```

```
[1 2 3]
(3,)
1D array: [1 2 3] Shape: (3,)
2D array: [[1]
 [2]
 [3]] Shape: (3, 1)
```

Array Indexing

In [34]:
```
import numpy as np

print("Matrix")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m)

print("\nAccess a single element")
print(m[0, 1])                          # Access an element
m[1, 1] = 100                           # Modify an element
print("\nModify a single element")
print(m)

print("\nAccess a subarray")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m[1, :])                          # Access a row (to 1D array)
print(m[1:2, :])                        # Access a row (to 2D array)
print(m[1:3, :])                        # Access a sub-matrix
print(m[1:, :])                         # Access a sub-matrix

print("\nModify a subarray")
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
v1 = np.array([1,1,1])
m[0] = v1
print(m)
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
v1 = np.array([1,1,1])
m[:,0] = v1
print(m)
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
m1 = np.array([[1,1],[1,1]])
m[:2,:2] = m1
print(m)

print("\nTranspose a subarray")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m[1, :].T)                                # Notice the difference of the dime
print(m[1:2, :].T)
print(m[1:, :].T)
print(np.transpose(m[1:, :], axes=(1,0)))       # np.transpose() can be used to tra

print("\nReverse the order of a subarray")
print(m[1, ::-1])                               # Access a row with reversed order

# Boolean array indexing
# Given a array m, create a new array with values equal to m
# if they are greater than 2, and equal to 0 if they less than or equal to 2
m = np.array([[1, 2, 3], [4, 5, 6]])
m[m > 2] = 0
print("\nBoolean array indexing: Modify with a scaler")
print(m)

# Given a array m, create a new array with values equal to those in m
# if they are greater than 0, and equal to those in n if they less than or equal 0
m = np.array([[1, 2, -3], [4, -5, 6]])
n = np.array([[1, 10, 100], [1, 10, 100]])
```

```
n[m > 0] = m[m > 0]
print("\nBoolean array indexing: Modify with another array")
print(n)
```

```
Matrix
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Access a single element
2

Modify a single element
[[  1   2   3]
 [  4 100   6]
 [  7   8   9]]

Access a subarray
[4 5 6]
[[4 5 6]]
[[4 5 6]
 [7 8 9]]
[[4 5 6]
 [7 8 9]]

Modify a subarray
[[1 1 1]
 [4 5 6]
 [7 8 9]]
[[1 2 3]
 [1 5 6]
 [1 8 9]]
[[1 1 3]
 [1 1 6]
 [7 8 9]]

Transpose a subarray
[4 5 6]
[[4]
 [5]
 [6]]
[[4 7]
 [5 8]
 [6 9]]
[[4 7]
 [5 8]
 [6 9]]

Reverse the order of a subarray
[6 5 4]

Boolean array indexing: Modify with a scaler
[[1 2 0]
 [0 0 0]]

Boolean array indexing: Modify with another array
[[  1   2 100]
 [  4  10   6]]
```

Array Dimension Operation

In [35]:
```
import numpy as np

print("Matrix")
m = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
print(m, m.shape)

print("\nReshape")
re_m = m.reshape(1,2,2)  # Add one more dimension at first.
print(re_m, re_m.shape)
re_m = m.reshape(2,1,2)  # Add one more dimension in middle.
print(re_m, re_m.shape)
re_m = m.reshape(2,2,1)  # Add one more dimension at last.
print(re_m, re_m.shape)

print("\nStack")
m1 = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
m2 = np.array([[1, 1], [1, 1]]) # Create a 2x2 array.
print(np.stack((m1,m2)))

print("\nConcatenate")
m1 = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
m2 = np.array([[1, 1], [1, 1]]) # Create a 2x2 array.
print(np.concatenate((m1,m2)))
print(np.concatenate((m1,m2), axis=0))
print(np.concatenate((m1,m2), axis=1))
```

```
Matrix
[[1 2]
 [3 4]] (2, 2)

Reshape
[[[1 2]
  [3 4]]] (1, 2, 2)
[[[1 2]]

 [[3 4]]] (2, 1, 2)
[[[1]
  [2]]

 [[3]
  [4]]] (2, 2, 1)

Stack
[[[1 2]
  [3 4]]

 [[1 1]
  [1 1]]]

Concatenate
[[1 2]
 [3 4]
 [1 1]
 [1 1]]
[[1 2]
 [3 4]
 [1 1]
 [1 1]]
[[1 2 1 1]
 [3 4 1 1]]
```

Math Operations on Array

Element-wise Operations

In [36]:
```python
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
print(a * 3)                                          # Scalar multiplication
print(a / 2)                                          # Scalar division
print(np.round(a / 2))
print(np.power(a, 2))
print(np.log(a))
print(np.exp(a))

b = np.array([[1, 1, 1], [2, 2, 2]], dtype=np.float64)
print(a + b)                                          # Elementwise sum
print(a - b)                                          # Elementwise difference
print(a * b)                                          # Elementwise product
print(a / b)                                          # Elementwise division
print(a == b)                                         # Elementwise comparison
```

```
[[ 3.  6.  9.]
 [12. 15. 18.]]
[[0.5 1.  1.5]
 [2.  2.5 3. ]]
[[0. 1. 2.]
 [2. 2. 3.]]
[[ 1.  4.  9.]
 [16. 25. 36.]]
[[0.         0.69314718 1.09861229]
 [1.38629436 1.60943791 1.79175947]]
[[  2.71828183   7.3890561   20.08553692]
 [ 54.59815003 148.4131591  403.42879349]]
[[2. 3. 4.]
 [6. 7. 8.]]
[[0. 1. 2.]
 [2. 3. 4.]]
[[ 1.  2.  3.]
 [ 8. 10. 12.]]
[[1.  2.  3. ]
 [2.  2.5 3. ]]
[[ True False False]
 [False False False]]
```

Broadcasting

In [37]:
```
# Note: See https://numpy.org/doc/stable/user/basics.broadcasting.html
#       for more details.
import numpy as np
a = np.array([[1, 1, 1], [2, 2, 2]], dtype=np.float64)
b = np.array([1, 2, 3])
print(a*b)
```

```
[[1. 2. 3.]
 [2. 4. 6.]]
```

Sum and Mean

In [38]:
```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("Sum of array")
print(np.sum(a))                # Sum of all array elements
print(np.sum(a, axis=0))        # Sum of each column
print(np.sum(a, axis=1))        # Sum of each row

print("\nMean of array")
print(np.mean(a))               # Mean of all array elements
print(np.mean(a, axis=0))       # Mean of each column
print(np.mean(a, axis=1))       # Mean of each row
```

```
Sum of array
21
[5 7 9]
[ 6 15]

Mean of array
3.5
[2.5 3.5 4.5]
[2. 5.]
```

Vector and Matrix Operations

In [39]:
```python
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[1, 1], [1, 1]])
print("Matrix-matrix product")
print(a.dot(b))                 # Matrix-matrix product
print(a.T.dot(b.T))

x = np.array([3, 4])
print("\nMatrix-vector product")
print(a.dot(x))                 # Matrix-vector product

x = np.array([1, 2])
y = np.array([3, 4])
print("\nVector-vector product")
print(x.dot(y))                 # Vector-vector product
```

```
Matrix-matrix product
[[3 3]
 [7 7]]
[[4 4]
 [6 6]]

Matrix-vector product
[11 25]

Vector-vector product
11
```

Matplotlib

`Matplotlib` is a plotting library. We will use it to show the result in this assignment.

In [40]:
```python
%config InlineBackend.figure_format = 'retina' # For high-resolution.
%matplotlib inline

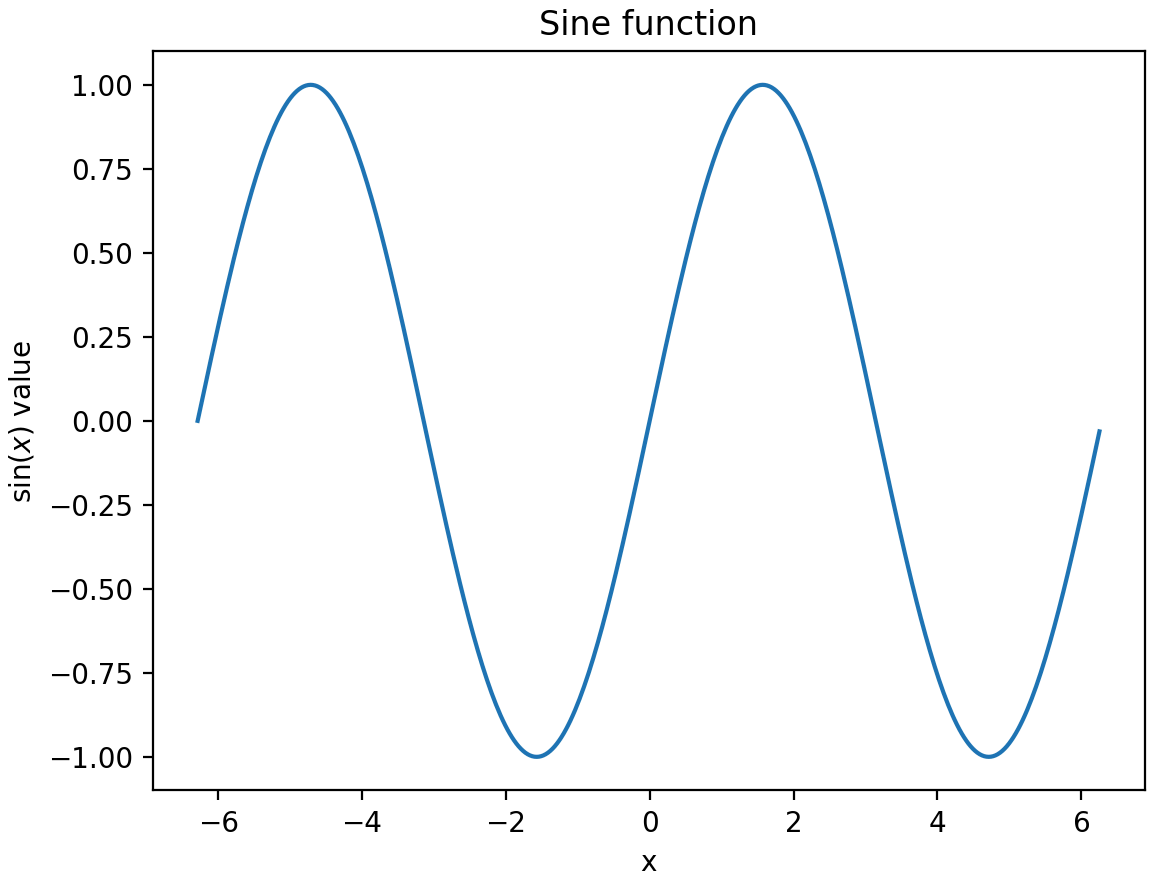
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-2., 2., 0.01) * np.pi
plt.plot(x, np.sin(x))
plt.xlabel('x')
plt.ylabel('$\sin(x)$ value') # '$...$' for a LaTeX formula.
```

```
plt.title('Sine function')

plt.show()
```

```
<>:10: SyntaxWarning: invalid escape sequence '\s'
<>:10: SyntaxWarning: invalid escape sequence '\s'
C:\Users\issac\AppData\Local\Temp\ipykernel_3804\420213150.py:10: SyntaxWarning: inv
alid escape sequence '\s'
  plt.ylabel('$\sin(x)$ value') # '$...$' for a LaTeX formula.
```



This brief overview introduces many basic functions from `NumPy` and `Matplotlib`, but is far from complete. Check out more operations and their use in documentations for NumPy and Matplotlib.

Problem 5: Vector Operations Using PyTorch (15 points)

In the previous section, we introduced vector operations using `NumPy`. `PyTorch`, as a popular deep learning framework, offers a similar interface as `NumPy`. In this problem, you will use the vector operations provided by `PyTorch`. You may need to refer to the official tutorial.

Suppose we have $W = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$, and $\mathbf{x}_1 = \begin{bmatrix} -1 \\ 1 \\ 2 \\ -3 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 5 \\ 1 \\ -3 \end{bmatrix}$.

The following code constructs W matrix and vector $\mathbf{b}, \mathbf{x}_1, \mathbf{x}_2$.

In [41]:
```
%pip install torch
```

```
Requirement already satisfied: torch in c:\users\issac\miniforge3\envs\dsc80\lib\sit
e-packages (2.9.1)
Requirement already satisfied: filelock in c:\users\issac\miniforge3\envs\dsc80\lib
\site-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\issac\miniforge
3\envs\dsc80\lib\site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy>=1.13.3 in c:\users\issac\miniforge3\envs\dsc80
\lib\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\issac\miniforge3\envs\dsc
80\lib\site-packages (from torch) (3.3)
Requirement already satisfied: jinja2 in c:\users\issac\miniforge3\envs\dsc80\lib\si
te-packages (from torch) (3.1.4)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\issac\miniforge3\envs\dsc80
\lib\site-packages (from torch) (2026.1.0)
Requirement already satisfied: setuptools in c:\users\issac\miniforge3\envs\dsc80\li
b\site-packages (from torch) (75.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\issac\miniforge3\envs
\dsc80\lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\issac\miniforge3\envs\dsc
80\lib\site-packages (from jinja2->torch) (2.1.5)
Note: you may need to restart the kernel to use updated packages.
```

In [42]:
```
import torch

W = torch.arange(1, 13).reshape(3, 4)
b = torch.tensor([2, 3, 1])
x_1 = torch.tensor([-1, 1, 2, -3])
x_2 = torch.tensor([5, 1, -3])
```

Now, use `PyTorch` to solve the following questions. You are supposed to use `PyTorch` **only, without** using `NumPy` or other libraries in Problem 5, including but not limited to converting `PyTorch` tensors to `NumPy` arrays or Python lists.

All your outputs are expected to be a `PyTorch` tensor (or a scalar number, if applicable).

(a) $W\mathbf{x}_1 + \mathbf{b}$

In [43]:
```
#### Write your code here ####
W @ x_1 + b
```

Out[43]:
```
tensor([ -3,  -6, -12])
```

(b) $W^\top \mathbf{x}_2 + \mathbf{x}_1$

In [44]:
```
#### Write your code here ####
W.T @ x_2 + x_1
```

Out[44]:
```
tensor([-18, -13,  -9, -11])
```

(c) $\mathbf{b} \circ \mathbf{x}_2$ ($\circ$ operator means element-wise product)

In [45]:
```
#### Write your code here ####
b * x_2
```

Out[45]:
```
tensor([10,  3, -3])
```

Suppose we have two more vectors $\mathbf{x}_3 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, \mathbf{x}_4 = \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix}$.

(d) Compute the dot product of $\mathbf{x}_3$ and $\mathbf{x}_4$.

In [46]:
```
# Construct x_3, x_4 and compute the dot product
### Write your code here ####
x_3 = torch.tensor([1, 0, -1])
x_4 = torch.tensor([2,-1, 1])

x_3.T @ x_4
```

Out[46]:
```
tensor(1)
```

(e) Suppose we have a new vector $\mathbf{c} = \begin{bmatrix} 3 \\ -1 \\ 2 \\ 1 \end{bmatrix}$. We would like to compute $W^\top \mathbf{x} + \mathbf{c}$ (where $\mathbf{x} = \mathbf{x}_3$ or $\mathbf{x}_4$). For efficiency, we want to calculate it in a batched manner.

Firstly, construct the vector $\mathbf{c}$, and stack $\mathbf{x}_3$ and $\mathbf{x}_4$ such that you get a 3×2 matrix, as such

$$X = \begin{bmatrix} \mathbf{x}_3 & \mathbf{x}_4 \end{bmatrix}$$

In [47]:
```
# Construct the vector c
c = torch.tensor([3, -1, 2, 1])

# Stack x_3 and x_4 to obtain the matrix X
# The shape of X should be 3x2
#### Write your code here ####

X = torch.stack([x_3, x_4], axis=1)
X
```

Out[47]:
```
tensor([[ 1,  2],
        [ 0, -1],
        [-1,  1]])
```

Now, you are supposed to calculate

$$W^\top X + \mathbf{c}$$

The result should be of shape 4×2. (Hint: To do it in a batched manner, you may want to convert the vector $\mathbf{c}$ to a matrix.)

In [48]:
```
#### Write your code here ####
W.T @ X + torch.stack([c, c], axis=1)
```

Out[48]:
```
tensor([[-5,  9],
        [-9,  7],
        [-6, 12],
        [-7, 13]])
```

(f) Create a tensor of size (3, 5) with random values drawn from a normal distribution (mean 0 and variance 1).

Use a built-in function in PyTorch to create this tensor. Print out the tensor and its shape.

In [49]:
```
# Create the tensor
#### Write your code here ####
T = torch.normal(0, 1, size=(3, 5))

# Print out this tensor
#### Write your code here ####
print(T)

# Print out the shape of the tensor
#### Write your code here ####
print(T.shape)
```

```
tensor([[-0.0939,  0.3083,  0.3665,  1.1858,  0.9840],
        [ 1.2547, -0.8559,  0.0447,  0.0449, -0.5501],
        [ 1.4024,  1.1090,  0.0525,  1.7720, -1.0890]])
torch.Size([3, 5])
```

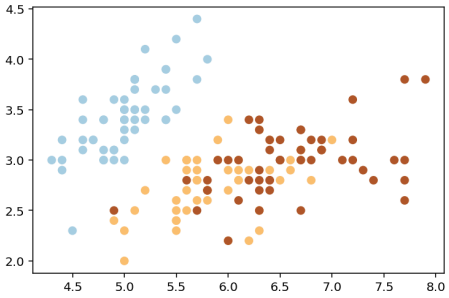
Problem 6 Data and Visualization (10 points)

In this section, we focus on feature visualization of Iris dataset from the `scikit-learn` package. First, we import some packages and load the Iris dataset

In [50]:
```
import matplotlib.pyplot as plt
from sklearn import datasets
from mpl_toolkits.mplot3d import Axes3D
```

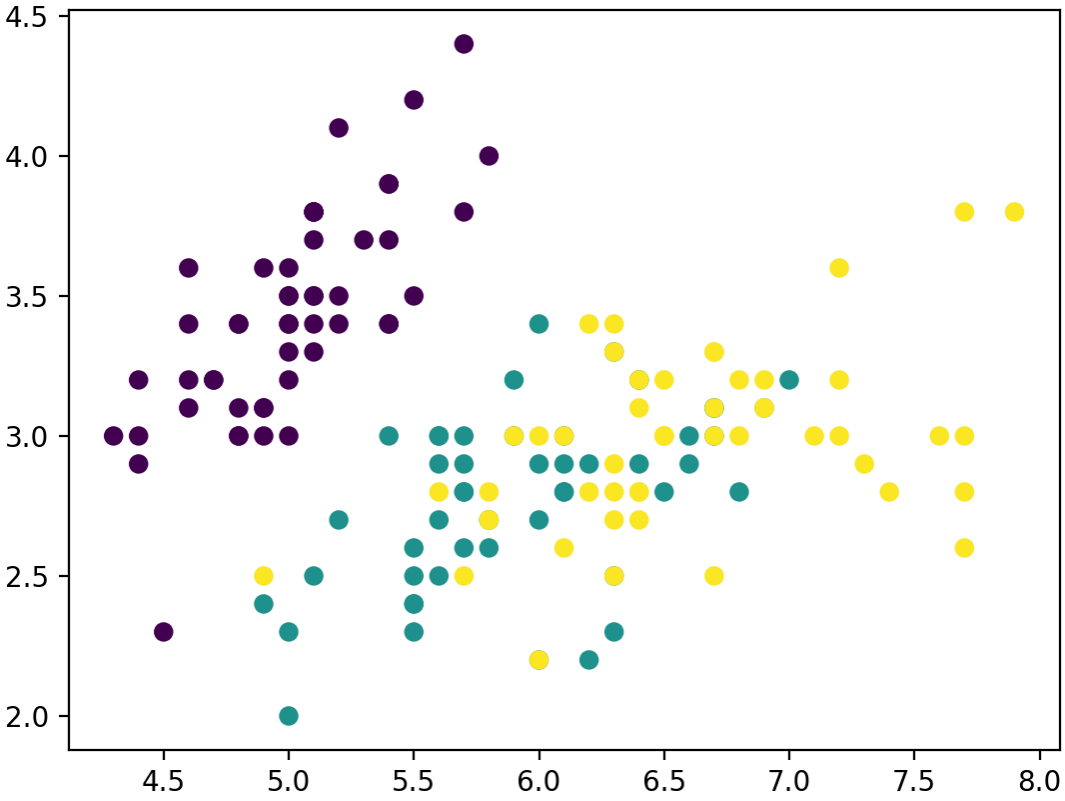
In [51]:
```
iris = datasets.load_iris()
X = iris.data
Y = iris.target
```

Now, plot a figure to visualize the first 2 feature dimensions. Your figure should look similar to the following

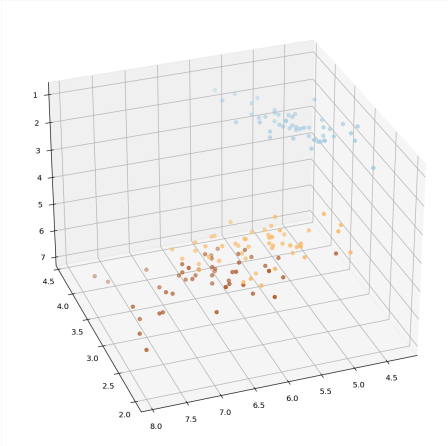


In [52]:
```
#### Write your code here ####
plt.scatter(X[:, 0], X[:, 1], c=Y)
```

Out[52]: <matplotlib.collections.PathCollection at 0x294705999a0>

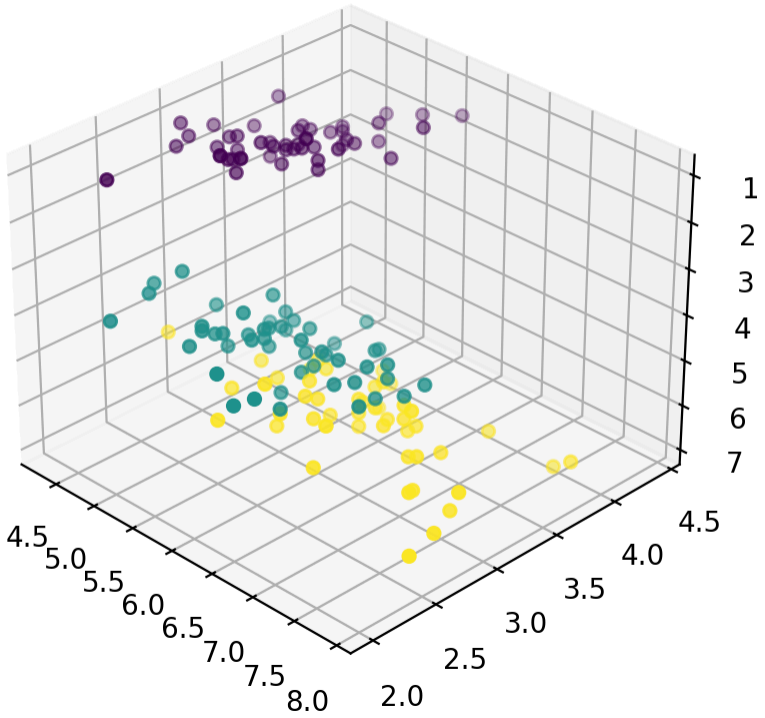


Plot a figure to visualize the first 3 feature dimensions. Your figure should look similar to the following



In [76]:
```
#### Write your code here ####
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.invert_zaxis()
ax.view_init(azim=-45)
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=Y)
```

Out[76]: `<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x294066bb860>`



Problem 7: Image Manipulation with NumPy (25 points)

In this problem we will use the image `bear.png`. Being a color image, this image has three channels, corresponding to the primary colors of red, green and blue.

(1) Read the image (using `Matplotlib`).

(2) Write a function to flip the original image **upside-down**. For this function, please only use **Array Indexing** to implement this function and **do not** use the functions (e.g. `np.flip()`) that directly flips the matrix.

(3) Next, convert the RGB image to grayscale image using the following equation

$$Y = 0.299R + 0.587G + 0.114B$$

The output image will be a single channel image of shape $H \times W$. Please ensure your shape for the output is correct.

(4) With the grayscale image obtained in (3), we define **blank pixels** $\mathcal{B} = \{(u, v) \mid I_{\text{grayscale}}(u, v) \geq 0.95\}$, where (u, v) is the pixel coordinate, and $I_{\text{grayscale}}(u, v)$ is the pixel value for that location in the grayscale image. Please set $I_{\text{color}}(u, v) = 0$ for all $(u, v) \in \mathcal{B}$, that is, set the pixel value to zero in the **color image** for all pixel coordinates in $\mathcal{B}$. Output the modified color image.

Hint: what do these **blank pixels** look like in the original color image? After setting their values to zero, these pixels should appear black.

(5) Finally, consider **4 images** you obtained: 1 original bear image, 1 from flipping upside-down, 1 from grayscale conversion, and 1 for removing the blank region. Using these 4 images, create one single image by tiling them together.

The output image will have 2×2 tiles making the shape of the final image $2H \times 2W \times 3$. The order in which the images are tiled does not matter. Show the tiled image.

Note:

1. For all the questions, please avoid loops by utilizing NumPy array indexing. Using loops will result in 20% point deduction for each question. A helpful guide on advanced NumPy array indexing is available here.
2. Please obtain the height and width for the image via functions such as `shape`. Hard-coding image size will result in 10% point deduction for each question.
3. In question (1)-(4), your function implementation should not alter the input image, otherwise it can affect the results for the subsequent questions. Also, you should only use the provided function parameters. For example, in (4), you should only use `grayscale_img` and `color_img`.

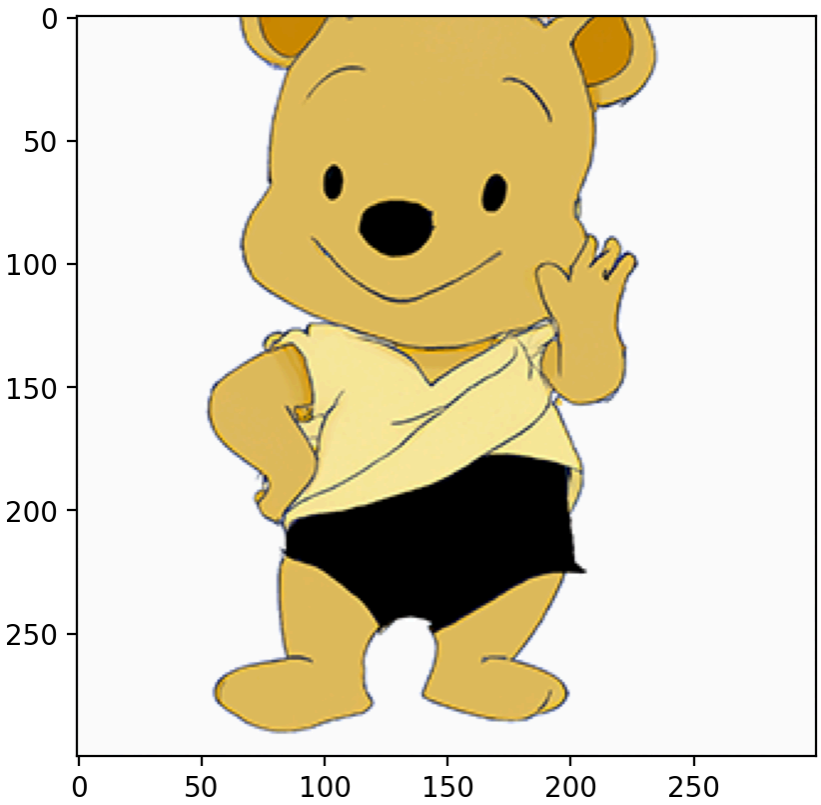
In [54]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [55]:
```
# (1) Read the image.
#### Write your code here. ####
```

```
img = plt.imread("../Images/bear.png")

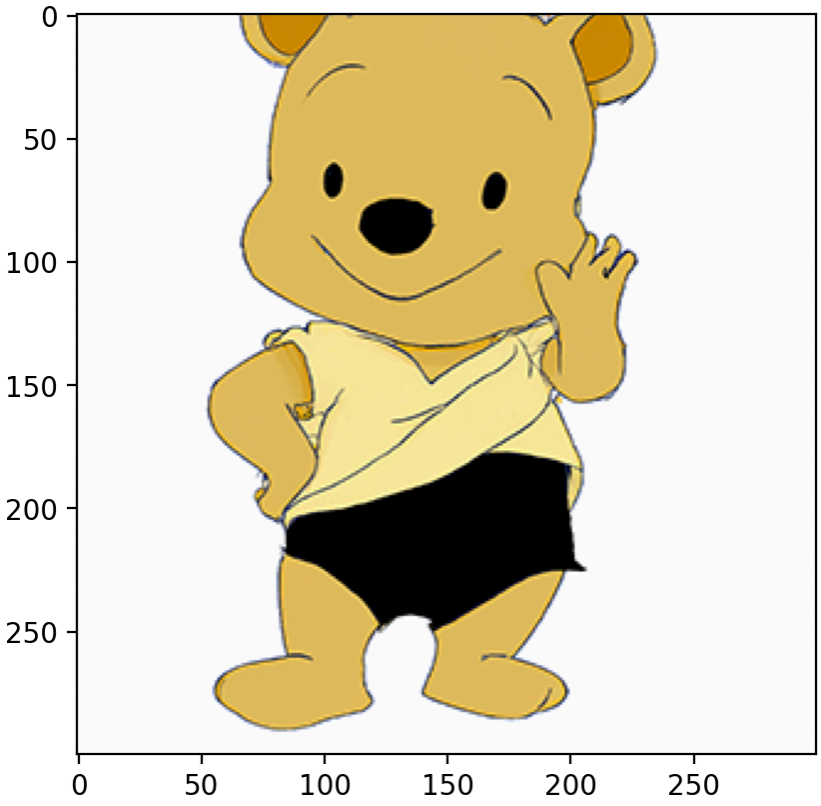
print(img.shape) # Show the shape of the image
plt.imshow(img) # Show the image after reading.
plt.show()
```

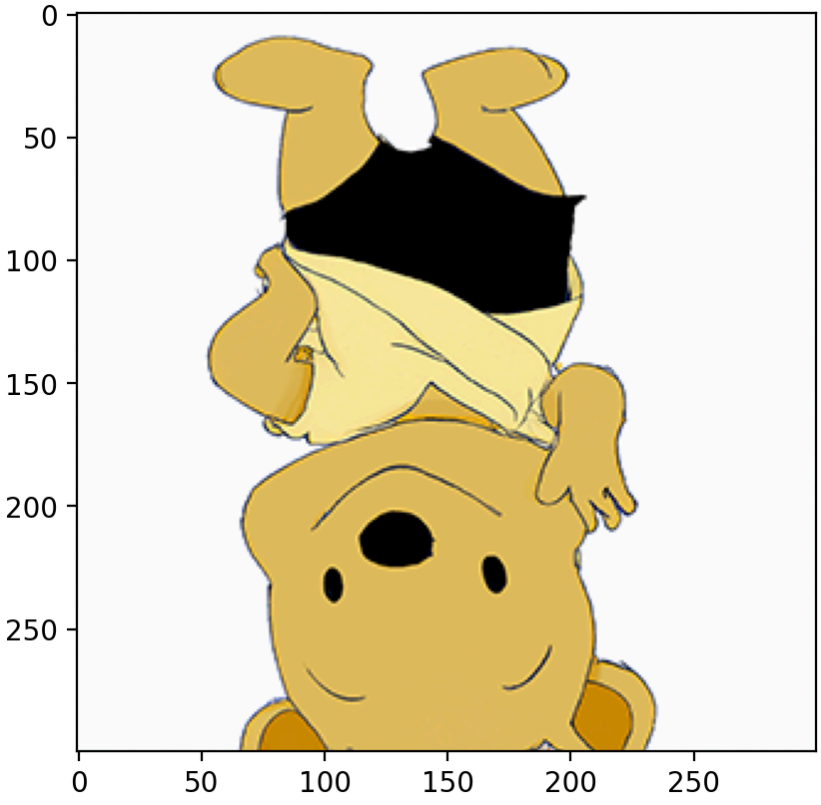
```
(300, 300, 3)
```



In [56]:
```
# (2) Flip the image upside-down.
def flip_img(img):
    """ Function to flip image upside-down.
    This function should return a H*W*3 array which is the flipped version of origi
    """
    #### Write your code here. ####
    return img[::-1]


plt.imshow(img)
plt.show()
flipped_img = flip_img(img)
plt.imshow(flipped_img)
plt.show()
```

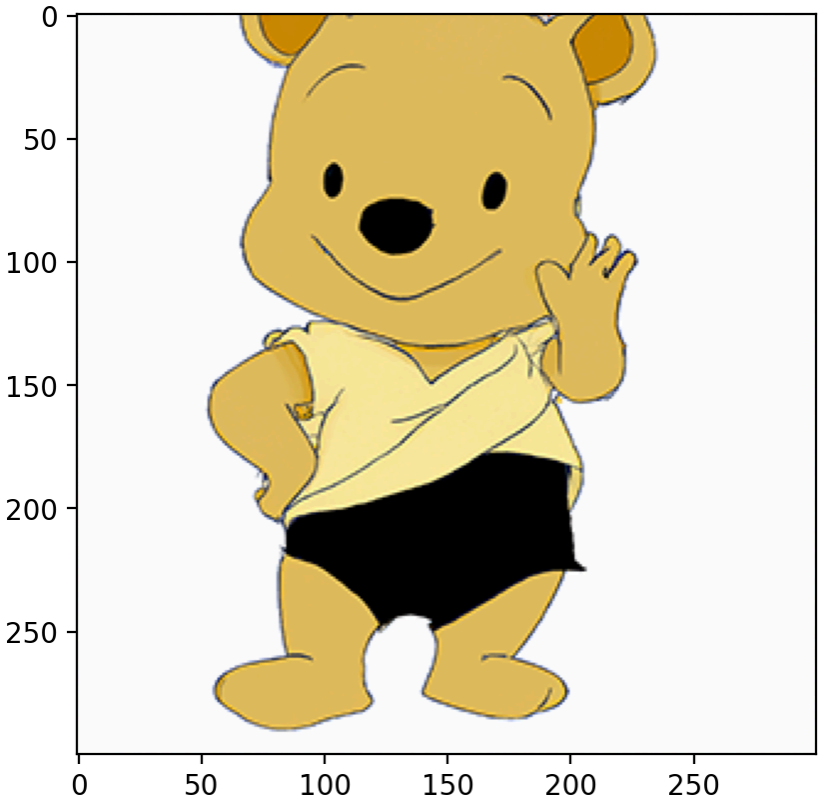


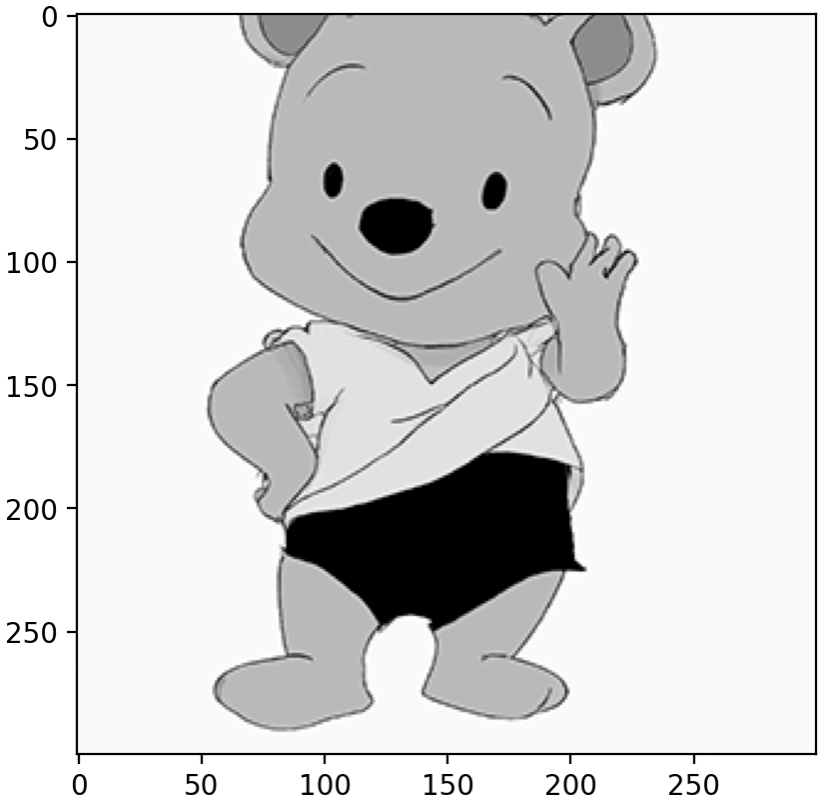


In [57]:
```
# (3) Convert RGB to grayscale
def grayscale(img):
    """ Function to convert RGB image to grayscale.
    This function should return a H*W array which is the grayscale version of origi
    """
```

```
    #### Write your code here. ####
    weights = np.array([0.299, 0.587, 0.114])
    return np.dot(img[..., :3], weights)


plt.imshow(img)
plt.show()
grayscale_img = grayscale(img)
plt.imshow(grayscale_img, cmap='gray')
plt.show()
```

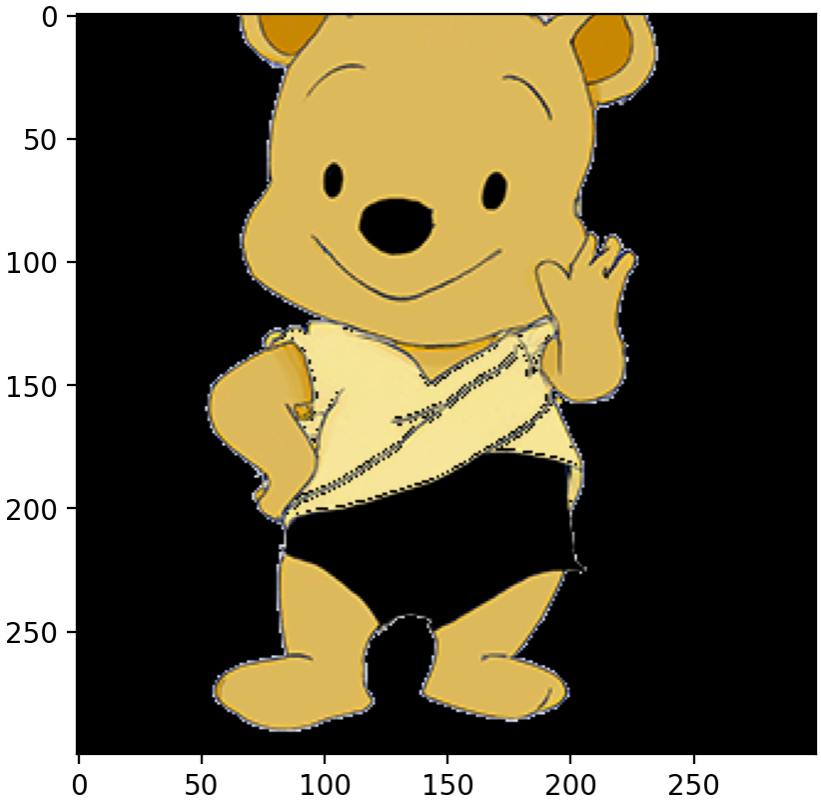




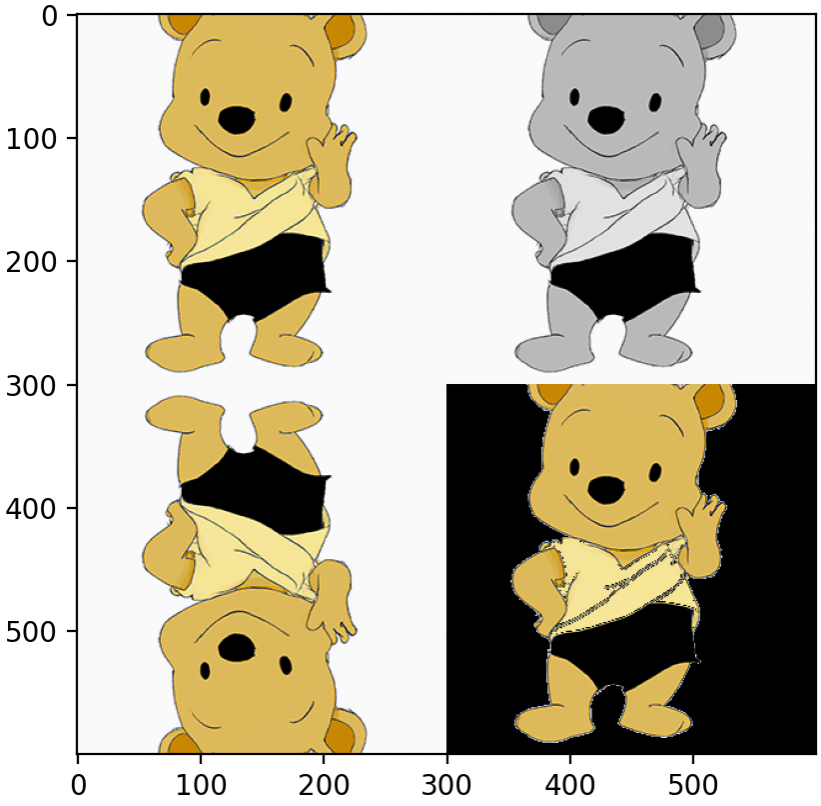
In [58]:
```
# (4) Set all the blank pixels to black (zero value) in the color image.
# Blank pixels are pixels whose values are >= 0.95 in the grayscale image.

def mask_blank_region(grayscale_img, color_img):
    """Function to mask (set value to zero) the blank pixels in the color image.
    Your implementation should not modify the original image. You can achieve this
    """
    #### Write your code here. ####
    masked = color_img.copy()
    masked[grayscale_img >= 0.95] = 0
    return masked

masked_img = mask_blank_region(grayscale_img, img)
plt.imshow(masked_img)
plt.show()
```



In [85]:
```
# (5) Write your code here to tile the four images and make a single image.
# You can use the img, flipped_img, grayscale_img, masked_img to represent the four
# After tiling, please display the tiled image.
#### Write your code here. ####
new_gray = np.stack((grayscale_img,) * 3, axis=-1)
images = [np.concat([img, flipped_img]), np.concat([new_gray, masked_img])]
plt.imshow(np.concat(images, axis=1))
plt.show()
```


0
100
200
300
400
500
0
100
200
300
400
500